

- 一. 认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。(15 分)
1. 无多人协同开发经历
 2. Git 设计“暂存—提交”两步，是为了在“工作区里杂乱、尚未定型的修改”和“历史中精确、稳定的提交”之间加入一个暂存区 (Index)。git add 先把本次真正想提交的文件或文件片段放入暂存区，git commit 再把暂存区内容生成一次提交；这样开发者可以在提交前审查、挑选甚至拆分改动，使每个提交只做一件事，便于回滚、review、git bisect 和 cherry-pick。同时暂存区也是 Git 构造提交 tree、处理合并冲突和缓存文件状态的核心结构，因此多这一步换来的是更干净、可控、可维护的版本历史。
 3. git branch 默认只列出本地分支，并在当前所在分支前用 * 标记；而 git branch -a (即 --all) 会同时列出本地分支和所有远程跟踪分支，远程分支通常显示为 remotes/origin/分支名，因此前者适合查看本地工作分支，后者适合了解远程仓库有哪些分支、准备基于远程分支创建或切换本地分支时使用。

- 二. 使用此仓库建立个人仓库，完成 main.c 文件中的 TODO 部分并进行一次 commit。

实验过程中一开始进程比较顺利，先是修改了 main.c，再以 change1.1 为信息暂存并 commit，但是后续将 commit 的结果同步到 github 上事出现报错，一开始是网络超时，更换网络后反复跳出来验证指纹。询问 ai 后发现是 wsl 的 git 和 windows 自带的 git 冲突了，问题出在 vs code 没有转换到 wsl 的远程上，所以我把原本的 lab0 文件都删了，再通过 git 重新 clone 到 wsl 里，这样只用 wsl 的 git 就不会冲突，最后顺利提交了 main.c 的 change1.1 版本到 GitHub 上。

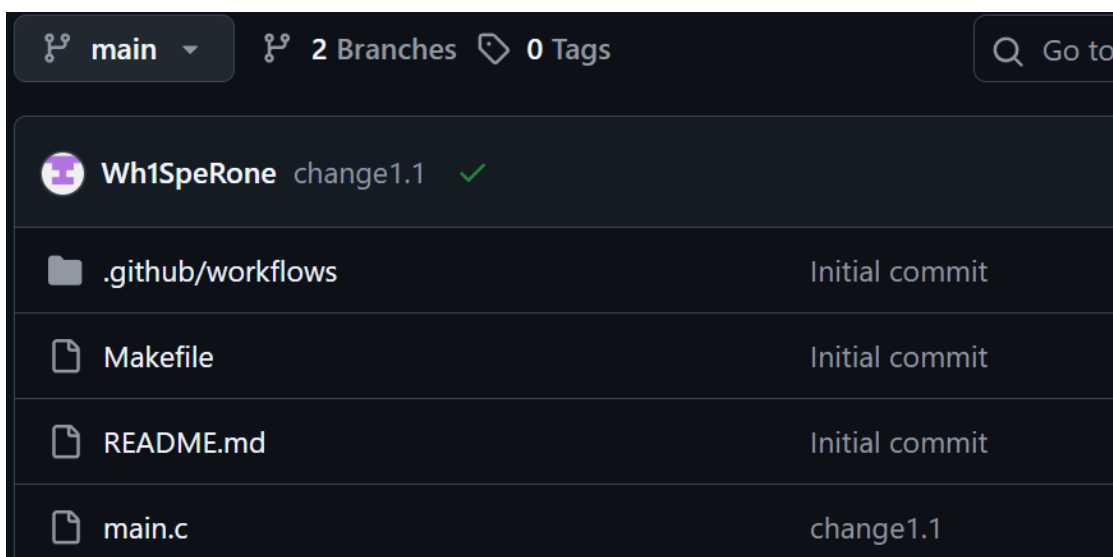
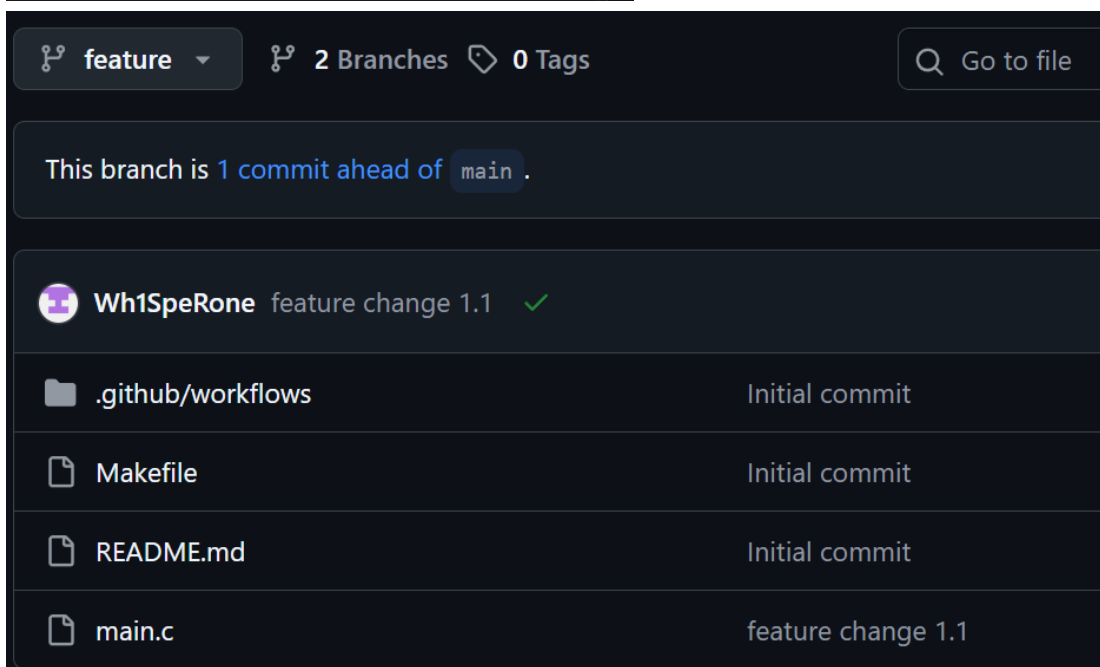
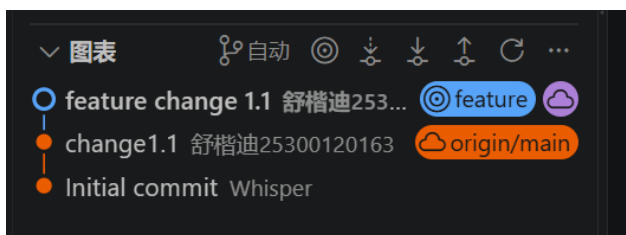
- 三. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。

我阅读了 commit message 规范和 git flow 分支控制，以下是我的理解：

学习 Git 不只是学会把代码上传到 GitHub，它的核心是对代码做版本管控：个人开发时通过 commit 给代码留存快照，搭配规范的提交备注，方便改错回退、复盘开发过程，不用靠复制文件夹来备份；而 GitFlow 这类分支规范，则为多人协作提供了一套行事准则，通过划分 master、develop、feature 等职能不同的分支，让大家并行开发互不干扰，隔离开发、测试与线上稳定代码，再结合清晰的提交记录追溯改动，既能解决代码互相覆盖冲突的问题，也能生成变更日志，养成良好的工程习惯，不管是完成课程作业，还是以后参与团队项目，都是必备的基础能力。

- 四. 学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交。随后将 feature 分支 merge 到 main 分支 (即切换回 main 分支执行 git merge feature)，并处理发生的合并冲突。

我先用 git checkout -b feature 新建了分支 feature 且转移到了 feature 分支，并对 feature 分支下的 main.c 文件进行了修改，commit 后同步到了 github 上



在 main 中的 main.c 中，我将输出的字符串改为了"Enjoy it.\n"

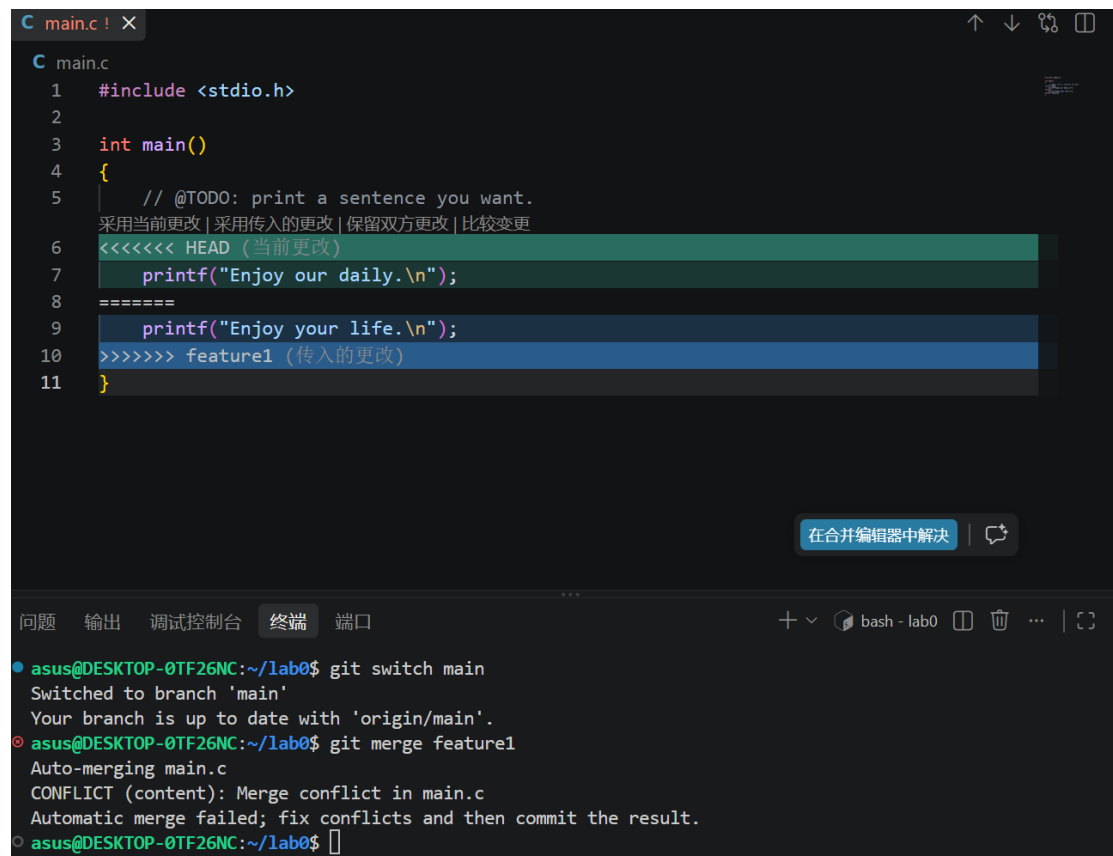
在 feature 中的 main.c 中，我将输出的字符串改为了"Enjoy that.\n"

但是执行合并操作时，并没有报错而是合并成功了

```
● asus@DESKTOP-0TF26NC:~/lab0$ git merge feature
Updating 4b85ee0..f17b6c9
Fast-forward
 main.c | 2 +
 1 file changed, 1 insertion(+), 1 deletion(-)
```

询问 ai 后发现是因为 feature 分支和 main 的进度不一样，feature 分支的进度更快，且 feature 分支是在 main 分支的基础上建立的，所以合并时即使两个文件不一样，git 也直接选用了 feature 分支中的文件

之后我又新建了分支 feature1，并对其中的 main.c 文件进行了修改，同时对 main 中的 main.c 文件也进行了修改，使两个分支中的同一行代码，在相同的任务进度下进行了不同的修改，从而使合并出现冲突



The screenshot shows a VS Code editor with a file named `main.c` open. The file content is as follows:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<<< HEAD (当前更改)
7      printf("Enjoy our daily.\n");
8      =====
9      printf("Enjoy your life.\n");
10     >>>>>> feature1 (传入的更改)
11 }
```

Below the editor, the terminal shows the following commands and output:

```
● asus@DESKTOP-0TF26NC:~/lab0$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
● asus@DESKTOP-0TF26NC:~/lab0$ git merge feature1
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
● asus@DESKTOP-0TF26NC:~/lab0$
```

所以当两个分支在非 fast-forward 的情况下修改了同一文件的同一位置，就会出现冲突，Git 无法自动合并

后面根据 vs code 的提示直接选择了以 main 下的 main.c 文件进行 merge 并提交和同步到

github 上了

 **main** ▾

 **3** Branches

 **0** Tags



 **Wh1SpeRone** Merge branch 'feature1' ✓



.github/workflows

Initial commit



Makefile

Initial commit



README.md

Initial commit



main.c

change 1.2